

Plano de Aula

Plano de Aula - Módulo 3: Recursos de Hardware (4 semanas | 16h)

Objetivo Geral:

Capacitar os alunos a utilizar e integrar recursos de hardware em aplicativos móveis, como Bluetooth, GPS, Wi-Fi, acelerômetro e multimídia (câmera e áudio), desenvolvendo aplicações que interajam com esses componentes.

Semana 1 - Introdução aos Recursos de Hardware

Objetivo: Apresentar conceitos fundamentais sobre a integração de hardware em aplicativos móveis, abordando Bluetooth, GPS e Wi-Fi.

Conteúdo Programático:

- Visão geral dos sensores e recursos de hardware disponíveis em dispositivos móveis.
- Configuração e permissões para uso de hardware em aplicativos móveis.
- Introdução ao Bluetooth: conexões, permissões e envio de dados.
- Uso de GPS: obtenção de localização e mapeamento.
- Conexão e monitoramento de redes Wi-Fi.

Metodologia:

- Aula expositiva com demonstração prática.
- Atividade prática: criar um aplicativo simples que obtenha a localização do usuário e exiba em um mapa.

Avaliação:

- Entrega da atividade prática.
-

Semana 2 - Implementação de Funcionalidades com Acelerômetro

Objetivo: Explorar o uso do acelerômetro para capturar e interpretar movimentos do dispositivo.

Conteúdo Programático:

- Conceitos básicos sobre sensores de movimento e acelerômetro.
- Obtendo dados do acelerômetro e interpretando valores.
- Criando interações com base nos movimentos do dispositivo.

Metodologia:

- Aula prática guiada.
- Desenvolvimento de um mini projeto: criar um aplicativo que detecta a inclinação do dispositivo e altera a interface com base nisso.

Avaliação:

- Implementação e apresentação do mini projeto.
-

Semana 3 - Uso de Recursos Multimídia - Câmera e Áudio

Objetivo: Ensinar os alunos a integrar funcionalidades de captura de imagem e gravação de áudio em aplicativos móveis.

Conteúdo Programático:

- Acesso à câmera para captura de imagens e vídeos.
- Gravação e reprodução de áudio.
- Manipulação de arquivos multimídia dentro do aplicativo.

Metodologia:

- Desenvolvimento prático de um aplicativo que permite tirar fotos e gravar áudios.
- Atividade prática: criar um aplicativo de anotações multimídia, onde o usuário pode adicionar imagens e gravações de voz.

Avaliação:

- Demonstração da funcionalidade implementada.
 - Relatório sobre o funcionamento do aplicativo.
-

Semana 4 - Projeto Final e Consolidação do Conteúdo

Objetivo: Consolidar os conhecimentos adquiridos por meio do desenvolvimento de um projeto que integre os recursos abordados.

Conteúdo Programático:

- Revisão geral dos conceitos aprendidos.
- Planejamento e desenvolvimento de um projeto integrador utilizando Bluetooth, GPS, Wi-Fi, acelerômetro e multimídia.

Metodologia:

- Desenvolvimento de um aplicativo completo em equipe.
- Apresentação dos projetos desenvolvidos.

Avaliação:

- Avaliação do projeto final com critérios de funcionalidade, interface e inovação.
 - Autoavaliação e feedback entre os alunos.
-

Recursos Didáticos:

- Ambiente de desenvolvimento mobile (Flutter).
- Dispositivos móveis ou emuladores.
- Documentação oficial e materiais complementares.

Critérios de Avaliação:

Link para LiveShared => 08/10 -

<https://prod.liveshare.vsengsaas.visualstudio.com/join?8E0D2B151DB725F1A52B8FBA75F7C337A408>

Plano de Ensino - Semana 1

Semana 1: Introdução aos Recursos de Hardware (4h)

Objetivo:

Apresentar conceitos fundamentais sobre a integração de hardware em aplicativos móveis, abordando Bluetooth, GPS e Wi-Fi, além de demonstrar a implementação prática desses recursos utilizando Flutter.

1. Conceitos Teóricos

1.1 Visão Geral dos Sensores e Recursos de Hardware em Dispositivos Móveis

Os dispositivos móveis modernos possuem diversos sensores e recursos de hardware que permitem interação avançada com o ambiente. Alguns dos mais utilizados incluem:

- **Bluetooth:** Comunicação sem fio entre dispositivos próximos.
- **GPS (Global Positioning System):** Determinação da localização geográfica do usuário.
- **Wi-Fi:** Conexão com redes sem fio para acesso à internet e comunicação local.
- **Acelerômetro e Giroscópio:** Detecção de movimento e orientação do dispositivo.
- **Sensores de proximidade e luz:** Controle de tela e ajuste de brilho conforme ambiente.

1.2 Configuração e Permissões para Uso de Hardware em Aplicativos Mobile

Para utilizar recursos de hardware em Flutter, é necessário definir permissões no arquivo de configuração do projeto.

No Android (AndroidManifest.xml):

No arquivo android/app/src/main/AndroidManifest.xml, adicione

```
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_COARSE_LOCATION"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
```

No iOS (Info.plist):

No arquivo ios/Runner/Info.plist, adicione:

```
<key>NSBluetoothAlwaysUsageDescription</key>
<string>O aplicativo precisa de acesso ao Bluetooth.</string>
<key>NSLocationWhenInUseUsageDescription</key>
<string>O aplicativo precisa acessar sua localização para exibir no mapa.</string>
<key>NSLocationAlwaysUsageDescription</key>
<string>O aplicativo precisa acessar sua localização em segundo plano.</string>
```

2. Demonstração de Aplicação em Flutter

2.1 Implementação do GPS (Localização do Usuário)

Para obter a localização do usuário em Flutter, utilizamos o pacote geolocator:

Instalação:

```
flutter pub add geolocator
```

Permissões:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION"/>
```

Código Exemplo:

<https://docs.google.com/document/d/1-CfC0MjMM6Kg-113cJGEUaxDEukcYsn0FqYM6CHey0U/edit?tab=t.8s4ypcsrfku5>

2.2 Implementação de Conexão Wi-Fi

Para monitorar a conexão Wi-Fi, utilizamos o pacote connectivity_plus:

Instalação:

```
flutter pub add connectivity_plus
```

Permissões:

```
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
```

Código Exemplo:

<https://docs.google.com/document/d/1-CfC0MjMM6Kg-113cJGEUaxDEukcYsn0FqYM6CHey0U/edit?tab=t.8s4ypcsrfku5>

2.3 Implementação de Conexão Bluetooth.

Vamos fazer uma demonstração básica do uso de Bluetooth em Flutter utilizando o pacote flutter_blue_plus, que é um dos mais utilizados atualmente para escanear, conectar e se comunicar com dispositivos Bluetooth Low Energy (BLE).

Instalação:

```
flutter pub add flutter_blue_plus
```

Permissões:

```
<uses-permission android:name="android.permission.BLUETOOTH" />  
<uses-permission android:name="android.permission.BLUETOOTH_ADMIN" />  
<uses-permission android:name="android.permission.BLUETOOTH_SCAN"  
  android:usesPermissionFlags="neverForLocation"/>  
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>  
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
```

Adicione dentro da tag <application>:

```
<uses-library android:name="org.apache.http.legacy" android:required="false"/>
```

Código Exemplo:

<https://docs.google.com/document/d/1-CfC0MjMM6Kg-113cJGEUaxDEukcYsn0FqYM6CHey0U/edit?tab=t.8s4ypcsrfku5>

3. Situação de Aprendizagem: App Registro de Localização com Mapa

Funcionalidades:

- Mostra o mapa com tiles do OpenStreetMap
- Obtém a localização atual do usuário com geolocator
- Armazena pontos visitados com data e hora
- Exibe marcadores personalizados no mapa
- Lista as localizações registradas

Dependências no pubspec.yaml

dependencies:

flutter:

 sdk: flutter

geolocator: ^10.1.0

flutter_map: ^6.1.0

latlong2: ^0.9.0

intl: ^0.18.1

Permissões Android

No android/app/src/main/AndroidManifest.xml, adicione:

```
<uses-permission android:name="android.permission.ACCESS_FINE_LOCATION" />
<uses-permission
android:name="android.permission.ACCESS_COARSE_LOCATION" />
```

Código da Aplicação

[10. RecursoHardware](#)

Resultado Esperado

- Ao iniciar o app, ele tenta obter sua localização.
- Ao clicar no botão de localização (📍), ele registra sua posição.
- A localização é mostrada no mapa com marcador.
- As posições anteriores ficam salvas e visíveis em uma lista.

Exercícios

1: Obter a Localização Atual

Objetivo: Aprender a usar o geolocator para obter a posição do usuário.

2: Obter Nome da Rede Wi-Fi

Objetivo: Mostrar o nome da rede Wi-Fi conectada usando wifi_info_flutter.

3: Escanear Dispositivos Bluetooth

Objetivo: Utilizar o flutter_blue para escanear dispositivos Bluetooth próximos.

4: Marcar Ponto no Mapa com OpenStreetMap

Objetivo: Usar o flutter_map com geolocator para marcar localização no mapa.

Desafio

Plano de Ensino - Semana 2

3. Image Picker - Captura de Imagem

O **image_picker** é um **plugin do Flutter** que permite capturar imagens e vídeos usando a câmera do dispositivo ou selecionar arquivos da galeria. Ele é amplamente utilizado em aplicativos que necessitam de interação com **multimídia**, como redes sociais, aplicativos de compartilhamento de fotos e apps de edição de imagens.

Funcionamento do image_picker

O plugin funciona acessando a câmera ou a galeria do dispositivo para capturar ou selecionar imagens e vídeos. Ele fornece uma interface simplificada para essas operações, permitindo ao desenvolvedor recuperar arquivos de mídia de maneira fácil e eficiente.

Principais Métodos:

O image_picker possui dois métodos principais:

Capturar uma imagem com a câmera

```
final XFile? image = await _picker.pickImage(source: ImageSource.camera);
```

- A câmera é aberta.
- O usuário tira uma foto.
- A imagem é retornada no formato XFile.

Selecionar uma imagem da galeria

```
final XFile? image = await _picker.pickImage(source: ImageSource.gallery);
```

- O usuário escolhe uma imagem da galeria.
- A imagem é carregada e disponibilizada no app.

Capturar um vídeo

```
final XFile? video = await _picker.pickVideo(source: ImageSource.camera);
```

- O usuário grava um vídeo.
- O vídeo é salvo e retorna o caminho do arquivo.

Selecionar múltiplas imagens

```
final List<XFile>? images = await _picker.pickMultilImage();
```

- Permite que o usuário escolha várias imagens da galeria.

Passo a Passo para Usar o image_picker no Flutter

- Adicionar a Dependência

```
flutter pub add image_picker
```

- Depois, execute:

```
flutter pub get
```

-
- **Configurar Permissões**

Android (AndroidManifest.xml)

No arquivo android/app/src/main/AndroidManifest.xml, adicione:

```
<uses-permission android:name="android.permission.CAMERA"/>
```

```
<uses-permission android:name="android.permission.READ_EXTERNAL_STORAGE"/>
```

```
<uses-permission android:name="android.permission.WRITE_EXTERNAL_STORAGE"/>
```

iOS (Info.plist)

No arquivo ios/Runner/Info.plist, adicione:

```
<key>NSCameraUsageDescription</key>
```

```
<string>Este aplicativo precisa acessar a câmera</string>
```

```
<key>NSPhotoLibraryUsageDescription</key>
```

```
<string>Este aplicativo precisa acessar a galeria de fotos</string>
```

Exemplo de Uso do Image_Picker

- **Resumo**

O image_picker facilita a captura de imagens e vídeos.

Suporte para **câmera e galeria**.

Pode ser usado para **selecionar múltiplas imagens**.

Requer **permissões no Android e iOS**.

Ideal para apps de **compartilhamento de mídia e edições de fotos**.

4. Atividade Prática

Objetivo: um aplicativo Flutter que permite ao usuário tirar fotos, armazená-las em uma galeria local e exibir a localização onde a foto foi tirada ao clicar nela.

Funcionalidades

Captura de fotos com a câmera

Armazena as fotos em uma **galeria local**

Registra a **localização GPS** ao tirar a foto

Exibe **data, hora e localização** ao clicar na imagem i

<https://docs.google.com/document/d/1-CfC0MjMM6Kg-113cJGEUaxDEukcYsn0FqYM6CHey0U/edit?tab=t.0>

5. Avaliação

1. Entrega da atividade prática, avaliando:

- Implementação correta das permissões.
- Uso da API de localização.
- Interface clara e funcional.

Situação de Aprendizagem Somativa

Situação de Aprendizagem - Avaliação Somativa em Flutter

Objetivos da Avaliação

- **Desenvolver habilidades práticas** no uso do framework Flutter, aplicando conceitos de desenvolvimento mobile nativo e integração com Firebase.
- **Demonstrar competências técnicas** em autenticação de usuários, uso de APIs externas (como geolocalização, QR Code, NFC, ou biometria) e gerenciamento de dados em tempo real.
- **Aplicar boas práticas** de desenvolvimento mobile, incluindo organização de código, interface de usuário (UI), experiência do usuário (UX).
- **Desenvolver a capacidade de solução de problemas** reais no contexto de aplicativos móveis, simulando um cenário de aplicação prática como registro de ponto, controle de acesso,

Tema 1 - App de Registro de Ponto com Geolocalização e Biometria

- **Descrição:** Um aplicativo que permite ao funcionário registrar seu ponto de trabalho quando estiver a até 100 metros do local de trabalho. A autenticação pode ser feita via NIF/email e senha ou utilizando reconhecimento facial via biometria.
- **Funcionalidades:**
 - Autenticação por NIF/email e senha ou reconhecimento facial.
 - Verificação de localização usando geolocalização.
 - Armazenamento do registro de ponto com data, hora e localização.
 - Integração com Firebase para autenticação e armazenamento dos dados.

Prazos e Cronograma

SESI B - 16/10 -

SESI A - 17/10 -

Castello B - 21/10

Castello A - 22/10

Entregáveis

Os alunos devem entregar os seguintes itens ao final da avaliação:

1. **Aplicativo Mobile em Flutter:**
 - Aplicativo funcional com todas as funcionalidades implementadas conforme o tema escolhido.
 - Código-fonte organizado e bem estruturado no formato de projeto Flutter.
 2. **Relatório de Implementação:**
 - Descrição técnica das funcionalidades implementadas e decisões de design.
 - Especificação do uso de APIs externas e integração com Firebase.
 - Explicação dos desafios encontrados e como foram resolvidos.
 3. **Documentação de Instalação e Uso:**
 - Passo a passo para rodar o aplicativo localmente.
 - Instruções para configurar Firebase e APIs utilizadas (geolocalização, NFC, biometria, etc.).
-

Critérios de Avaliação por Competências

A avaliação será baseada nas seguintes competências, cada uma com um peso específico para o total da nota:

- **Competência Técnica (40%):**
 - Implementação correta das funcionalidades principais do aplicativo (ex: geolocalização, autenticação, QR code, biometria).
 - Integração adequada com Firebase (autenticação, armazenamento de dados em tempo real).
 - Utilização correta de pacotes externos (geolocalização, biometria, NFC).
 - Estrutura de código clara, organizada e reutilizável.
- **Interface e Usabilidade (20%):**
 - Interface de usuário amigável e responsiva.
 - Implementação de uma experiência de usuário (UX) que facilite o uso do aplicativo.
 - Uso de boas práticas de design em Flutter (uso de Widgets adequados, navegação entre telas).
- **Criatividade e Solução de Problemas (20%):**
 - Capacidade de adaptar o projeto a um problema real.
 - Inovação na implementação de recursos adicionais ou melhorias além do proposto.
- **Documentação (20%):**
 - Documentação clara e concisa explicando a implementação do projeto.

- Instruções de uso do aplicativo e como configurar o ambiente de desenvolvimento.